

Типы данных, структуры данных, управляющие конструкции



Актуализация

- Какие типы данных вы использовали в Pascal? В Python? В 1C?
- Чем integer отличается от real?
- Что такое массив?
- Как записать условие «если число больше 0» на Python? На Pascal? На 1C?

Типы и структуры данных, управляющие конструкции. Различия

Типы

Какие?

Структуры

Как
организованы

Упр. конст.

Какой
порядок?

Структуры и типы данных, управляющие конструкции. Различия

- Тип данных — **что за значение** хранится и какие операции с ним допустимы.
- Структура данных — **как значения организованы** в памяти для эффективного доступа и изменения.
- Управляющие конструкции — **в каком порядке** выполняются инструкции программы.

Типы данных

Категория	Примеры
Числовые	целые (int), вещественные (float, double), рациональные, комплексные
Логические	boolean (True/False)
Символьные и строковые	char, string, str
Дата и время	date, time, datetime
Бинарные	bytes, bytearray
Специальные	None, null, nil, Неопределено

Примитивы в PascalABC, Python, 1C, Java

Целые числа

PascalABC	Python	1C	Java
shortint smallint integer longint Int64 byte word (16-бит) longword, cardinal uint64 BigInteger	int	Число (нету разделения на дробные и целые)	byte short int long

Примитивы в PascalABC, Python, 1C, Java

Дробные
числа

Символьные,
строковые

Логические
Спец.

PascalABC	Python	1C	Java
real double single decimal	float	Число (нету разделения на дробные и целые)	float double
char string	str (нету разделения на char и string)	Строка (нету разделения как и в python)	char, String
boolean	bool	Булево	boolean
Nullable	NoneType	Неопределено	null (безымянный тип)

В Python нету примитивов



QuarterObvious • 2 г. назад

Даже "примитивный" тип `int` - это класс с кучей свойств и атрибутов:

```
print(dir(int))
```

```
['abs', 'add', 'and', 'bool', 'ceil', 'class', 'delattr', 'dir', 'divmod', 'doc', 'eq', 'float', 'floor', 'floordiv', 'format',  
'ge', 'getattribute', 'getnewargs', 'gt', 'hash', 'index', 'init', 'init_subclass', 'int', 'invert', 'le', 'lshift', 'lt', 'mod',  
'mul', 'ne', 'neg', 'new', 'or', 'pos', 'pow', 'radd', 'rand', 'rdivmod', 'reduce', 'reduce_ex', 'repr', 'rfloordiv',  
'rshift', 'rmod', 'rmul', 'ror', 'round', 'rpow', 'rrshift', 'rshift', 'rsub', 'rtruediv', 'rxor', 'setattr', 'sizeof', 'str',  
'sub', 'subclasshook', 'truediv', 'trunc', 'xor', 'as_integer_ratio', 'bit_length', 'conjugate', 'denominator',  
'from_bytes', 'imag', 'numerator', 'real', 'to_bytes']
```

```
__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc__',
```

```
class int(object):
```

```
    """
```

```
    int([x]) -> integer
```

```
    int(x, base=10) -> integer
```

Convert a number or string to an integer, or return 0 if no arguments are given. If x is a number, return x.__int__(). For floating point numbers, this truncates towards zero.

If x is not a number or if base is given, then x must be a string, bytes, or bytearray instance representing an integer literal in the given base. The literal can be preceded by '+' or '-' and be surrounded by whitespace. The base defaults to 10. Valid bases are 0 and 2-36. Base 0 means to interpret the base from the string as an integer literal.

```
>>> int('0b100', base=0)
```

```
4
```

```
.....
```

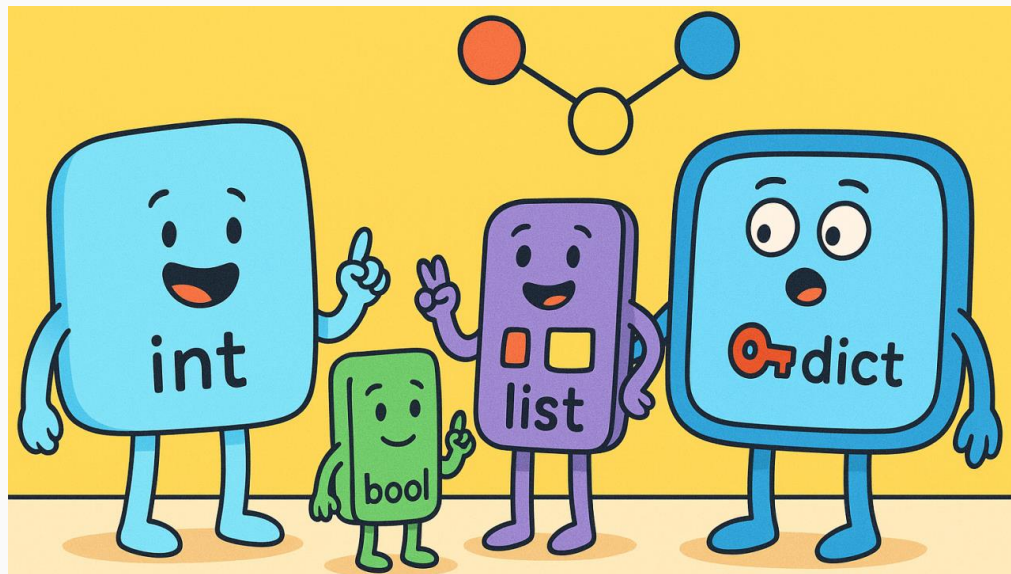
Структуры данных

1. Это способ организации и хранения информации в памяти компьютера, который позволяет эффективно получать к ней доступ и изменять её
2. Это информация, связанная определённым образом
3. Это объект-контейнер, в котором хранится информация. Данные внутри контейнера структурированы по особой системе — она различается в зависимости от вида структуры.
4. Программная единица, позволяющая хранить и обрабатывать однотипные и/или логически связанные данные. Для добавления, поиска, изменения и удаления данных структура данных предоставляет некоторый набор функций, составляющих её интерфейс.

.....

Типы структур данных

- Линейные
- Нелинейные
- Ассоциативные



Основные структуры данных (сводка)

Структура	Тип	Особенность
Массив	Линейная	Фиксированный размер, доступ по индексу
Список	Линейная	Быстрое добавление/удаление
Стек	Линейная	LIFO (Last-in-First-out)
Очередь (Queue)	Линейная	FIFO (First-in-First-out)
Дек (Deque)	Линейная	Два конца
Множество	Ассоциативная	Уникальные элементы
Словарь	Ассоциативная	Ключ → значение
Хеш-таблица	Ассоциативная	Быстрый поиск по ключу
Дерево	Нелинейная	Иерархия
Куча (не область памяти)	Нелинейная	Приоритетная очередь
Граф	Нелинейная	Связи между вершинами
Запись / класс	Составная	Разнотипные поля

Линейные

- Массив (Array) — непрерывная область памяти, доступ по индексу.
- Связный список (Linked List) — узлы связаны ссылками. Бывает односвязный, двусвязный, кольцевой.
- Стек (Stack) — принцип LIFO (последний пришёл — первый вышел).
- Очередь (Queue) — принцип FIFO (первый пришёл — первый вышел).
- Дек (Deque) — очередь с двумя концами.
- Список (List) — абстрактный тип, может быть реализован массивом или связным списком



Нелинейные

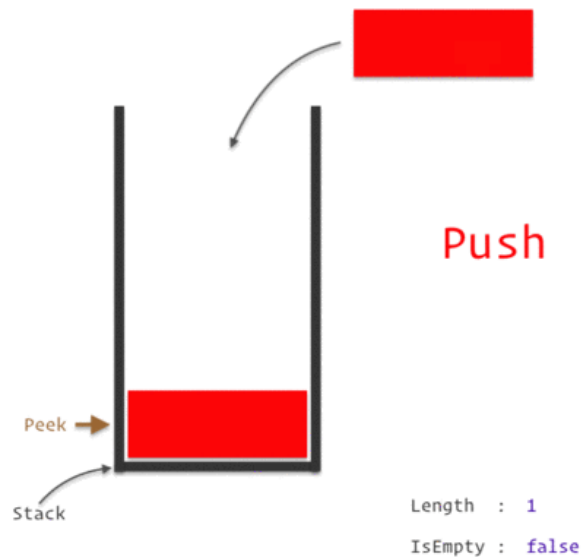
- Дерево (Tree) — иерархия узлов. Виды: бинарное дерево, BST, AVL, красно-чёрное, куча (heap), B-дерево, префиксное дерево (Trie).
- Граф (Graph) — набор вершин и рёбер. Бывает ориентированный, неориентированный, взвешенный.

Ассоциативные

- Множество (Set) — уникальные элементы, без порядка.
- Словарь / Map / Ассоциативный массив — пары «ключ — значение».
- Хеш-таблица (Hash Table) — реализация словаря на основе хеш-функции.

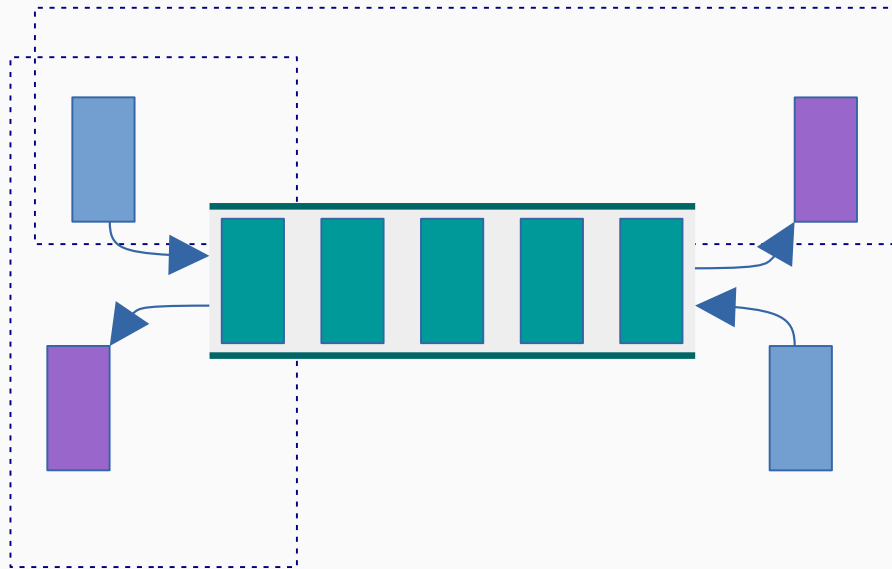
Stack

- Push – добавляет элементор
- Pop – удаляет элемент
- Peek – получает первый

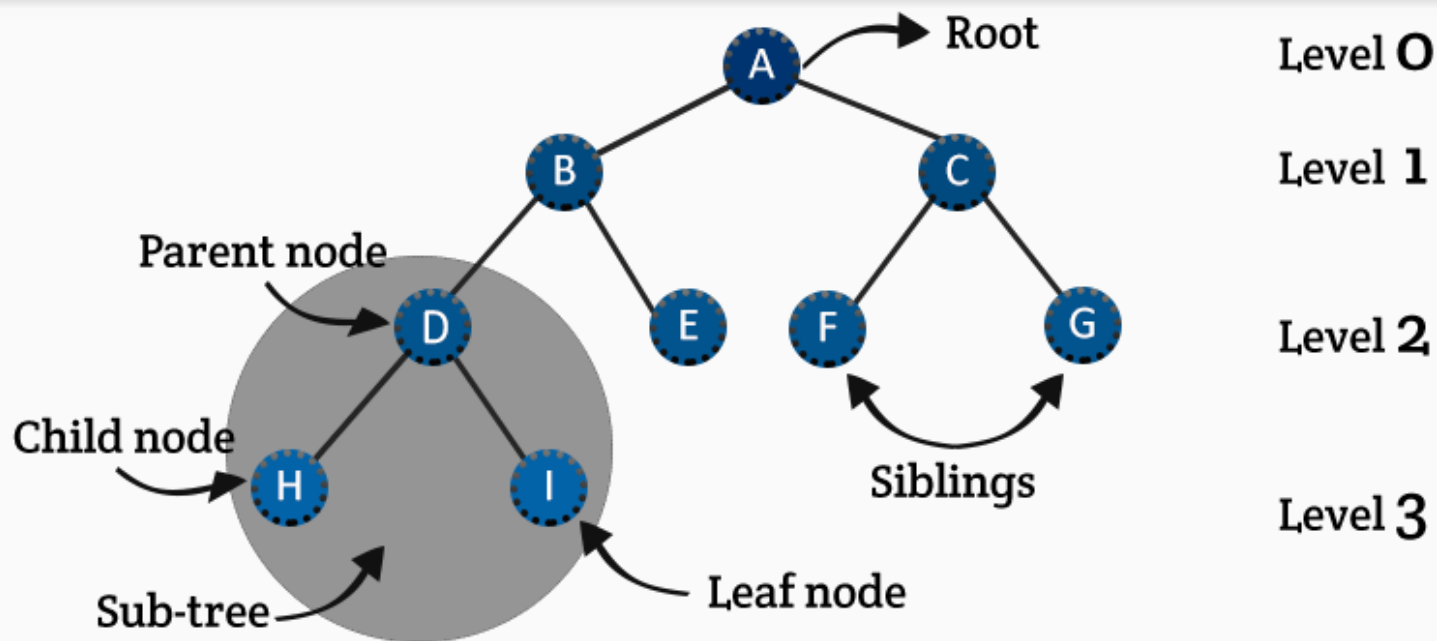


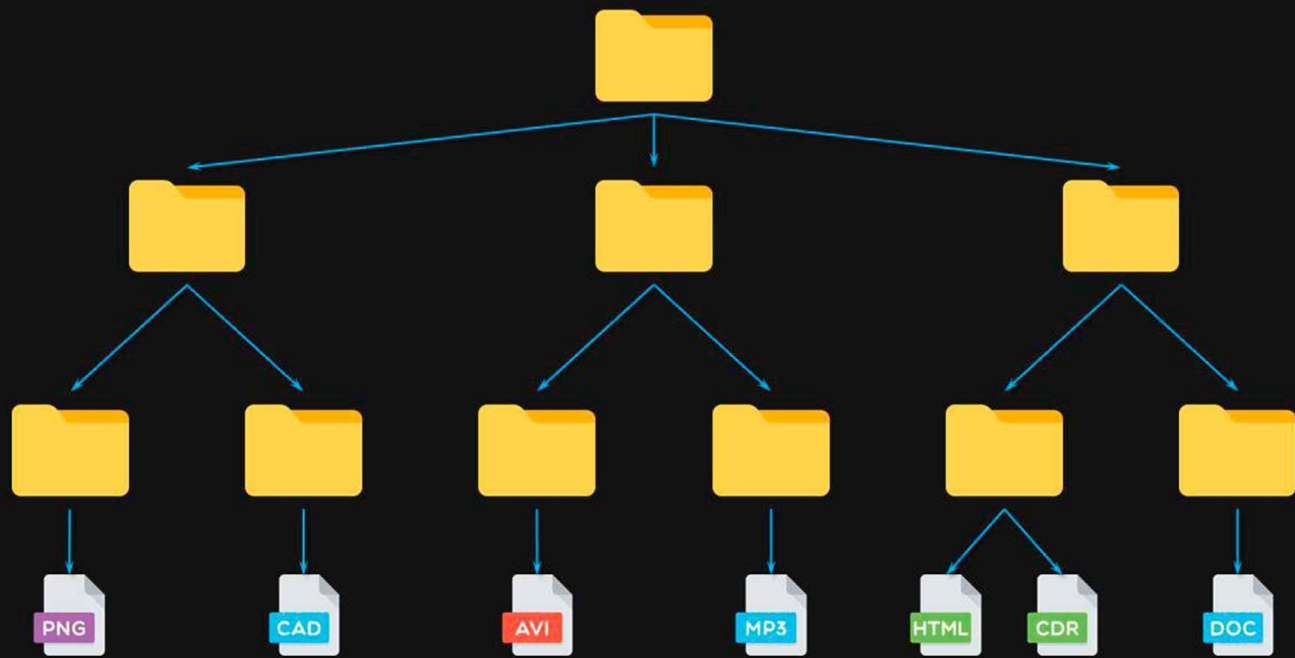
Deque (Double-ended-Queue)

- Позволяет добавлять элементы как в начало, так и в конец. Объединяет в себе функции стэка и очереди.

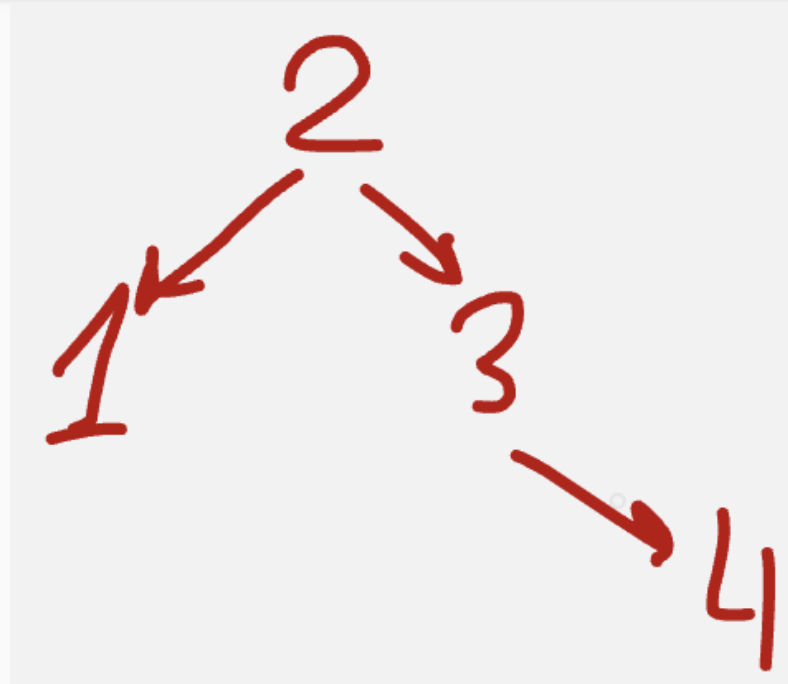


Tree data structure



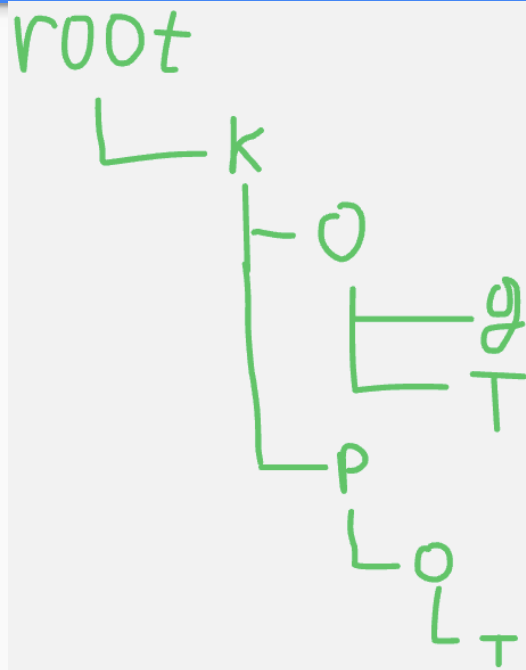


Самобалансирующееся дерево



Префиксное дерево (бор, trie, от retrieval)

- Автодополнение и подсказки (T9)
- Проверка орфографии и словари
- IP-маршрутизация и сети
- Поиск подстрок и биоинформатика
- Базы данных и поисковые движки
-



Массив

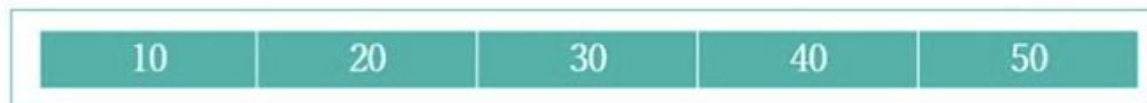
Diagram illustrating the declaration of an array:

`int arr[5] = {10, 20, 30, 40, 50};`

The components are labeled:

- `int`: Data type (indicated by an upward arrow from "Data type")
- `arr`: Name (indicated by a downward arrow from "Name")
- `5`: Size (indicated by an upward arrow from "Size")
- `{10, 20, 30, 40, 50}`: Elements (indicated by a horizontal line above the set)

Elements



Index

0

1

2

3

4

Управляющие конструкции

- Управляющие конструкции — это конструкции языка, которые определяют порядок выполнения инструкций. Без них программа была бы линейной: одна команда за другой. С ними появляется логика: ветвление, повторение, выбор.
- Их можно разделить на три группы:
 1. Ветвление — if, else, switch/case.
 2. Циклы — for, while, do-while, foreach.
 3. Операторы прерывания и продолжения — break, continue, return.

Ветвление (if / else)

Python	Pascal
<pre>if x > 0: print("Положительное") elif x < 0: print("Отрицательное") else: print("Ноль")</pre>	<pre>if x > 0 then writeln('Положительное') else if x < 0 then writeln('Отрицательное') else writeln('Ноль');</pre>
1C	Java
<pre>Если x > 0 Тогда Сообщить("Положительное"); ИначеЕсли x < 0 Тогда Сообщить("Отрицательное"); Иначе Сообщить("Ноль"); КонецЕсли;</pre>	<pre>if (x > 0) System.out.println("Положительное"); else if (x < 0) System.out.println("Отрицательное"); else System.out.println("Ноль");</pre>

Тернарный оператор

Язык	Код
Python	<code>status = "Совершеннолетний" if age >= 18 else "Несовершеннолетний"</code>
PascalABC	тернарного оператора нет, только if-else.
1C	<code>Статус = ?(Возраст >= 18, "Совершеннолетний", "Несовершеннолетний");</code>
Java	<code>String status = (age >= 18) ? "Совершеннолетний" : "Несовершеннолетний";</code>

switch / case / match

Python	Pascal
<pre>match day: case 1: print("Понедельник") case 2: print("Вторник") case _: print("Другой день")</pre>	<pre>case day of 1: writeln('Понедельник'); 2: writeln('Вторник'); else writeln('Другой день'); end;</pre>
1C	Java
аналога switch нет	<pre>switch (day) { case 1: System.out.println("Понедельник"); break; case 2: System.out.println("Вторник"); break; default: System.out.println("Другой день"); }</pre>

Управляющие конструкции (сводка)

Конструкция	Python	Java	Pascal	1C
Условие	if x > 0:	if (x > 0) {}	if x > 0 then	Если x > 0 Тогда
Иначе если	elif	else if	else if	ИначеЕсли
Тернарный	a if c else b	c ? a : b	нет	?(c, a, b)
Множественный выбор	match	switch	case	нет
Цикл с счётчиком	for i in range()	for (int i=0; ...)	for i := ... to ...	Для i = ... По ...
Цикл с условием	while	while	while	Пока
Постусловие	нет	do-while	repeat-until	нет
По коллекции	for item in	for (item : coll)	for item in	Для Каждого
Прервать	break	break	break	Прервать
Продолжить	continue	continue	continue	Продолжить
Выход из метода	return	return	exit	Возврат

for с счётчиком

Python	Pascal
<pre>for i in range(1, 6): print(i)</pre>	<pre>for i := 1 to 5 do writeln(i);</pre>
1C	Java
<pre>Для i = 1 По 5 Цикл Сообщить(i); КонецЦикла;</pre>	<pre>for (int i = 1; i <= 5; i++) { System.out.println(i); }</pre>

while

Python	Pascal
<pre>while x < 10: x += 1</pre>	<pre>while x < 10 do x := x + 1;</pre>
1C	Java
<pre>Пока x < 10 Цикл x = x + 1; КонецЦикла;</pre>	<pre>while (x < 10) { x++; }</pre>

do-while / repeat-until

Python	Pascal
нет do-while	repeat $x := x + 1$; until $x \geq 10$;
1C	Java
нет do-while	do { $x++$; } while ($x < 10$);

foreach (цикл по коллекции)

Python	Pascal
<pre>for item in items: print(item)</pre>	<pre>for item in items do writeln(item);</pre>
1C	Java
<pre>Для Каждого item Из items Цикл Сообщить(item); КонецЦикла;</pre>	<pre>for (String item : items) { System.out.println(item); }</pre>

Операторы прерывания и продолжения

Python	Pascal
<pre>for i in range(1, 10): if i == 5: break print(i)</pre>	<pre>for i := 1 to 9 do begin if i = 5 then break; writeln(i); end;</pre>
1C	Java
<pre>Для i = 1 По 9 Цикл Если i = 5 Тогда Прервать; КонецЕсли; Сообщить(i); КонецЦикла;</pre>	<pre>for (int i = 1; i < 10; i++) { if (i == 5) break; System.out.println(i); }</pre>

continue — перейти к следующей итерации

Python	Pascal
<pre>for i in range(1, 10): if i % 2 == 0: continue print(i) # только нечётные</pre>	<pre>for i := 1 to 10 do begin if i % 2 = 0 then Continue; writeln(i); end;</pre>
1C	Java
<pre>Для i = 1 По 9 Цикл Если i % 2 = 0 Тогда Продолжить; КонецЕсли; Сообщить(i); КонецЦикла;</pre>	<pre>for (int i = 1; i < 10; i++) { if (i % 2 == 0) continue; System.out.println(i); }</pre>

return — ВЫЙТИ ИЗ МЕТОДА

Python	Pascal
<pre>def find(arr, target): for i, value in enumerate(arr): if value == target: return i return -1</pre>	Следующий слайд
1C	Java
Функция Найти(Массив, Цель) Для i = 0 По Массив.Количество() - 1 Цикл Если Массив[i] = Цель Тогда Возврат i; КонецЕсли; КонецЦикла; Возврат -1; КонецФункции	<pre>public int find(int[] arr, int target) { for (int i = 0; i < arr.length; i++) { if (arr[i] == target) return i; } return -1; }</pre>

```
function Find(arr: array of Integer; target: Integer): Integer;  
var  
    i: Integer;  
begin  
    for i := 0 to Length(arr) - 1 do  
        begin  
            if arr[i] = target then  
                begin  
                    Result := i;  
                    Exit;  
                end;  
            end;  
        end;  
    Result := -1;  
end;
```

Метки в Java

Java позволяет помечать циклы и использовать break/continue с меткой
В Python, 1C, Pascal меток для break нет.

```
outer:
for (int d = 0; d < 3; d++) {
    for (int j = 0; j < 3; j++) {
        if (d == 1 && j == 1) break outer;
        System.out.println(d + " " + j);
    }
}
```